Reinforcement Learning Assignment: Markov Decision Processes

Group of 1 or 2

Deliverable: Written solutions + code + a short report + Presentation

Evaluation: Sep 23, Wednesday

Part 1 — Problem Setup

Consider a 4×4 Frozen Lake gridworld:

- **States:** 16 cells, $s \in \{0, 1, ..., 15\}$
- **Actions:** A = {Left, Down, Right, Up}
- **Dynamics:** With probability 0.25 the agent moves in the intended direction
- **Rewards:** −0.01 per step (living cost), +1 for reaching the goal, −1 for falling into a hole. Terminal states: goal and holes.
- **Discount factor:** $\gamma = 0.9$

Task: Draw the grid (mark hole and goal locations yourself), define the full MDP tuple (S, A, P, R, γ), and state whether this MDP is finite, episodic, and fully observed. Justify each answer in one or two sentences.

Part 2 — Policy Evaluation

Take the **uniform random policy** $\pi(a|s)=0.25$ for all s, a.

1. Write down the Bellman expectation equation for the value function under this policy.
2. Implement policy evaluation in Python using **iterative updates** (not matrix inversion — that's part 4). Show the value function converging: plot $||V_{k+1} - V_k||_\infty$ versus iteration k.
3. Manually compute $V^\pi(s)$ for **one** non-trivial state (e.g., a cell adjacent to a hole) for the first two iterations, and verify your code matches your hand calculation.
4. From V^π, compute the action-value function $Q^\pi(s,a)$ for every state-action pair.

Part 3 — Policy Iteration & Value Iteration

1. **Policy iteration:** Starting from the random policy, derive the optimal policy π^*. Show the policy at each improvement step — how many iterations does it take? Does the policy stabilize before the values fully converge?
2. **Value iteration:** Run value iteration and recover π^* greedily. Confirm it matches your policy iteration result.
3. **Analysis question:** For a gridworld, policy iteration usually converges in very few policy improvement steps, while value iteration needs many steps. Explain *why* using the contraction properties of the Bellman operators.

Part 4 — From Iterative to Exact Solution

1. Re-derive policy evaluation using **exact linear algebra**: set up the system $V^\pi = R^\pi + \gamma P^\pi V^\pi$ and solve via matrix inversion with NumPy. Compare with your iterative result from Part 2 — report the max absolute difference.
2. **Critical thinking:** Linear algebra gives an exact answer for a 16-state problem. Why is this approach infeasible for problems with large number of states? What property of RL algorithms makes them scalable?

Part 5 — Exploration Questions

Short written answers (3–5 sentences each):

1. What happens to the optimal policy as γ is 0 versus γ is 1? Demonstrate this by rerunning value iteration with $\gamma = 0.5$ and $\gamma = 0.999$.
2. Change the living reward from -0.01 to 0. Does the optimal policy change?
3. How would your code need to change if you **couldn't assume knowledge of P and R**? (This is the setup for the next lectures — model-free methods.)
